

Work in Progress: Efficient Readers-Writer Locks for the RTIC Framework

Anonymous Authors for review

Anonymous

Anonymous

Anonymous

anonymous@example.com

Anonymous Authors for review

Anonymous

Anonymous

Anonymous

anonymous@example.com

Anonymous Authors for review

Anonymous

Anonymous

Anonymous

anonymous@example.com

Abstract—The RTIC framework provides an executable model for concurrent applications as a set of static priority, run-to-completion jobs with shared resources. At run-time, the system is scheduled in compliance with Stack Resource Policy (SRP), which guarantees race- and deadlock-free execution for single-processor systems. While the original work on SRP allows for multi-unit resources, the RTIC framework uses a model that is constrained to single-unit resources.

We review the theoretical foundations of readers-writer locks in the context of SRP to show that they can be implemented in RTIC at zero cost when compared to existing binary-semaphore-based locks, while relaxing the constraints of the worst-case-blocking-time-based schedulability test. We provide declarative model for the implementation of code generation for RTIC, compatible with the ARM Cortex-M and RISC-V architectures.

Index Terms—Real-Time Systems, Stack Resource Policy, Readers-Writer Locks, RTIC, Rust

I. INTRODUCTION

The RTIC framework provides a Rust-language executable model for concurrent applications as a set of static priority, preemptive, run-to-completion jobs with shared resources. At run-time, the system is scheduled in compliance with Stack Resource Policy (SRP)[1]—an extension to Priority Ceiling Protocol (PCP)[2]—which guarantees a number of desirable features for single-processor scheduling. Features of SRP include race- and deadlock-free execution, bounded, single-context-switch-per-job blocking, prevention of multiple priority inversion, and simple, efficient, single-shared-stack execution.

The original theory for SRP[1] describes a scheduling policy for a system with multi-unit resources that can be used to implement binary semaphores, readers-writer locks, and general semaphores. RTIC—*however*—only implements a mutex based on the binary semaphore. Replacing the binary semaphore with a readers-writer lock, when applicable, lowers the estimate for blocking time, meaning that when using scheduling tests including worst-case blocking factors¹, more systems will pass the test.

The rationale for the current constrained implementation of RTIC is that a binary semaphore is sufficient to provide safe access to shared resources. Furthermore, in read-write situations where the highest priority contender for a resource is a

job of the writing type, a binary semaphore already provides similar schedulability to readers-writer locks under SRP.

However, in situations where the highest priority contender is not a write, a readers-writer lock would improve the response time of high-priority readers. Therefore, inclusion of the readers-writer lock in RTIC’s supported lock types would extend RTIC’s applicability across real-time systems with high-priority readers. This paper describes a declarative model of SRP-compliant readers-write locks that can be implemented in RTIC at no additional cost, when compared to a mutex based on a binary semaphore. General multi-unit resources are also of interest. However, an overhead-free implementation has not been identified, and is therefore left for future work.

Our contributions are:

- the observation and proof that for each read- and write-lock operation, it is possible to compute such a distinct ceiling value at compile time, that SRP compliant resource protection can be implemented in constant time,
- a declarative model for the implementation of a readers-writer lock in RTIC with no additional overhead when compared to the binary semaphore based mutex,
- the observation that the implementation aligns the SRP compliant readers-writer lock with the Rust aliasing model, allowing lock APIs to integrate seamlessly with Rust’s reference semantics.
- a description of target-independent code generation for readers-writer resources in RTIC.

II. PRIOR WORK

A. SRP-based scheduling

PCP describes a locking protocol for binary semaphores, for which priority inversion is bounded by execution time of the longest critical section of a lower-priority job. [2] PCP has been extended to apply to readers-writer resources[4], and multi-processor systems [5]. SRP extends single-core PCP and allows the use of both static and dynamic priority assignments, and multi-unit resources. [6] PCP and SRP-based methods remain of interest for hard real-time scheduling, as conventional OSes cannot provide bounded blocking suitable for real-time schedulability analysis. [6], [7]

¹E.g., the recurrent worst-case response time test [3] or the RM specific utilization factor test [4] for schedulability.

B. Rust aliasing guarantees

The Rust rules regarding *place* and *move* expressions², and *pointer*³ types require that any memory location referenced by the program is either shared for reading, or only accessible for writing from *one* code location concurrently, in absence of *interior mutability* and *raw pointer dereferences*⁴. To access the exceptions, an explicit, `unsafe` code block is always required. The alias rules can be extended to apply to other kinds of resources including hardware peripherals by modeling them as Zero-Sized Types (ZSTs). It should be carefully noted that certain hardware operations such as side-effectful reads from hardware-internal buffers should be considered writes from a software-and-concurrency point of view, and should be modelled as such in Rust (cf. `Read` trait in `embedded-io`⁵).

Rust’s alias rules coincide with the semantics of readers-writer locks. In other words, the interfaces of the lock may grant shared references to readers, while mutable references can be granted to writers, all the while conforming to Rust’s notion of safety. Both kinds of references should be scoped to match the duration that the lock is held.

C. RTIC

The RTIC framework is a Rust-first, free-and-open-source, real-time framework, rooted in research on modeling and implementation of (hard) real-time systems. Over the last decade, the first two major versions of RTIC—`cortex-m-rtic` and RTIC v2—have gained wide adoption with a combined download count of more than a million on `crates.io`. For the developer, RTIC provides a declarative, SRP-compliant, tasks-and-resources model as a thin, integrated DSL in the form of attributes applied on *items*⁶ in Rust code. RTIC also provides facilities for compile time analysis, code generation, and the zero-cost abstractions for implementing the concurrency model.

Supplemental to the mainline RTIC, a research prototype[9] of the framework has been developed to study—in a modular way—implementations of features including syntax extensions and extended source code analysis. In the research prototype, the compilation process is separated into a set of subsequent passes that gradually lower the DSL-augmented source artifact towards a plain Rust implementation, and then further into an executable. In Section VI, the modular research prototype is leveraged to sketch out the implementation of readers-writer resources.

III. BASELINE MODEL (SRP)

SRP assumes a fixed number of run-to-completion jobs running on a single core, sharing a fixed number of multi-unit resources. The maximum resource needs are assumed to be

known *a priori*. Jobs are assumed to request anything from zero to the full amount of a multi-unit resource.

In SRP, a job⁷ J will preempt another if its *preemption level* $\pi(J)$ is higher than the *system ceiling* $\bar{\Pi}$ and it’s the oldest and highest priority of any pending job. The preemption level of a job $\pi(J)$ is defined as any static function that satisfies

$$p(J') > p(J) \text{ and } J' \text{ arrives later} \Rightarrow \pi(J') > \pi(J). \quad (1)$$

For static priority assignments, $\pi(J) = p(J)$.

The system ceiling $\bar{\Pi}$ is defined as the maximum of current *resource ceilings*, which are values assigned to each resource that depend on their own, current availability. The resource ceiling $\lceil R \rceil$ must always be equal or higher than the preemption level of the running job, and all the preemption levels of jobs that might need R more than what is currently available. Formally, resource ceiling is any function that satisfies

$$\lceil R \rceil_{v_R} \geq \max(\{\pi(J_{\text{cur}})\} \cup \{\pi(J) \mid v_R < \mu_R(J)\}), \quad (2)$$

where J_{cur} is the currently executing job, v_R is the current availability of R and $\mu_R(J)$ is the maximum need of job J for R . Assuming the system has resources $R_i, i \in \{0, \dots, n\}$,

$$\bar{\Pi} = \max(\{\lceil R_i \rceil \mid i \in \{0, \dots, n\}\}). \quad (3)$$

From the definition, it follows that the system ceiling $\bar{\Pi}$ changes only when a resource is locked or unlocked or when a new job starts executing. When a lock on R is obtained, the system ceiling is updated to

$$\bar{\Pi}_{\text{new}} = \max(\bar{\Pi}_{\text{cur}}, \lceil R \rceil_{v_R}), \quad (4)$$

where $\bar{\Pi}_{\text{cur}}$ is the prior system ceiling, and $\lceil R \rceil_{v_R}$ is the ceiling of R corresponding to the remaining amount of unlocked R .

A. Readers-writer Resources

Readers-writer resources are a special case of multi-unit resources. In the SRP context, they can be modeled as an abstract resource with a count equaling the number of jobs accessing the resource, and writers consuming all units of the resource, while readers consume only one unit. This allows multiple readers but only one writer at a time. Generally, an infinite number of readers is allowed, but only a single write at any time.

IV. RTIC RESTRICTED MODEL

RTIC compiles programmer-defined and -prioritized jobs to interrupt handlers that get a corresponding, relative priority level. The jobs—now ISRs—are run preemptively, in priority order, by the hardware. The lock closures in the code are compiled into instructions that update the system ceiling to a value determined by the compiler, and restore it upon unlock. The targets supported by RTIC must have prioritized interrupts and support for interrupt masking. The interrupt masking is used to create a hardware implementation of the SRP defined system ceiling.

²[8, [expr.place-value.value-expr-kinds], [expr.move]]

³[8, [type.pointer.reference]]

⁴[8, [interior-mut], [type.pointer.raw.safety]]

⁵https://docs.rs/embedded-io/latest/embedded_io/trait.Read.html

⁶[8, [items]]

⁷The original theory distinguishes a job J and it’s execution or request $\mathcal{J}$. However, in this paper, only J is used, as with static priority jobs, this distinction is not necessary.

In RTIC so far, only single-unit resources have been allowed, as with them, the system ceiling needs to be updated to a single, compile-time known number for each resource. RTIC leverages this to implement near zero-cost locking. With each lock operation on a resource, the interrupts with a lower priority than the compile-time known number are disabled. The means of disabling the appropriate interrupts depend on the implementation target.

Formally, in RTIC, the preemption levels equal the priority: $\pi = p$, and the resource ceiling is defined as

$$\lceil R \rceil = \max(\{0\} \cup \{p(J) \mid v_R < \mu_R(J)\}), \quad (5)$$

where v_R is the current availability of R and $\mu_R(J)$ is the maximum need of job J for R . Inclusion of $p(J_{\text{cur}})$ is not needed due to hardware running the jobs in preemptive priority order, making it part of the system ceiling. [10]

With this set-up and using only single-unit resources, HW implements SRP compliant scheduling, when each lock operation on R raises the system ceiling to

$$\bar{\Pi}_{\text{new}} = \max(\bar{\Pi}_{\text{cur}}, \lceil R \rceil_0) \quad (6)$$

and upon unlock—at the end of the lock closure—the old value is restored. In combination with the Rust ownership system and compliance with SRP, controlled access to shared, single-unit resources is guaranteed.

A. ARM Cortex-M

On Cortex-M [11], [12], [13], external interrupts can be controlled and configured with the Nested Vectored Interrupt Controller (NVIC). Registers called NVIC_IPR control the priorities of the external interrupts.

The context of a preempted ISR is pushed to stack and restored automatically by the hardware. An ISR can be preempted safely while it is saving the context, increasing the responsiveness of high priority ISRs.

Depending on the architecture, interrupts can be masked either using the BASEPRI register, or if it's not implemented, the NVIC_ISER and NVIC_ICER registers. The BASEPRI register blocks interrupts of lower or equal priority than its value, but it can not block interrupts with maximum possible priority. The NVIC_ISER and NVIC_ICER registers bits enable or disable individual interrupts.

B. RISC-V

The base RISC-V ISA[14] does not require a sufficient mechanism for individually configurable preemption levels or threshold-based interrupt filtering. Instead, this mechanism is typically supplied through an interrupt controller specification. For instance, the CLIC[15] defines an adjustable interrupt threshold register (`mintthresh`) that can be used to filter interrupts by preemption level. For interrupt-specific priority and preemption controls, the CLIC defines the `clicintctl` register. On RISC-V, when multiple lines are pended, priority is used to determine which interrupt handler is to be dispatched first, while preemption level is used to control preemptability. [16]

V. SRP COMPLIANT READERS-WRITER LOCK

As already discussed, the current version of RTIC uses only single-unit resources. With multi-unit resources of the readers-writer type, there is still a single compile-time known number that the system ceiling needs to be raised to with each lock operation—just like in (6), but with a distinction of whether the lock is a read or a write lock. A formalization and a proof of the statements follows:

Theorem

Assuming (5) and $\pi = p$, SRP compliance is maintained when:

- 1) upon taking a read-lock of resource R is taken, the system ceiling $\bar{\Pi}$ is updated to

$$\bar{\Pi} = \max(\bar{\Pi}_{\text{cur}}, \lceil R \rceil_r) \quad (7)$$

where $\lceil R \rceil_r$ is the highest preemption level of jobs with write-access to R , and

- 2) upon taking a write-lock of resource R , the system ceiling $\bar{\Pi}$ changes to

$$\bar{\Pi} = \max(\bar{\Pi}_{\text{cur}}, \lceil R \rceil_w), \quad (8)$$

where $\lceil R \rceil_w$ is the highest preemption level of jobs that need R .

Proof

Assume the system has resources $R_1, \dots, R_n$ and their availability is $v_{R_1}, \dots, v_{R_n}$ before taking the lock. Assume the read or write lock operation concerns resource R_m , $m \in 1, \dots, n$. It can be shown that after the locking, the system ceiling is

$$\bar{\Pi} = \max(\{\bar{\Pi}_{\text{cur}}\} \cup \max\{\pi(J) \mid v'_{R_m} < \mu_{R_m}(J)\}), \quad (9)$$

where v'_{R_m} is the new availability of resource R_m .

Proof for (7) (read-lock):

After locking, either $v'_{R_m} \in \{1, \dots, n-1\}$ or $v_{R_m} = 0$.

In the former case, the condition $v'_{R_m} < \mu_{R_m}(J)$ in (9) corresponds to J having write access to R_m , proving (7).

In the latter case, (9) can be expanded to

$$\begin{aligned} \bar{\Pi} = \max(\{ & \bar{\Pi}_{\text{cur}} \} \\ & \cup \{\pi(J) \mid J \text{ has read access to } R_m\} \\ & \cup \{\pi(J) \mid J \text{ has write access to } R_m\}). \end{aligned} \quad (10)$$

Assuming the same job does not take several nested read locks, for there to be zero R_m after a read lock, the job must have preempted all other jobs that only read R_m while they were holding a lock on resource R_m . For that to be possible, the job has to be the highest priority job with read access to R_m , i.e.,

$$\pi(t_{\text{cur}}) = \max\{\pi(J) \mid J \text{ has read access to } R_m\} \quad (11)$$

Continuing from (10),

$$\begin{aligned}
& \stackrel{(11)}{\Rightarrow} \bar{\Pi} = \max(\{\bar{\Pi}_{\text{cur}}\} \cup \{\pi(t_{\text{cur}})\}) \\
& \cup \{\pi(J) \mid J \text{ has write access to } R_m\}) \\
& \stackrel{\bar{\Pi} \geq \pi(J_{\text{cur}})}{=} \max(\{\bar{\Pi}_{\text{cur}}\} \\
& \cup \{\pi(J) \mid J \text{ has write access to } R_m\}) \\
& = \max(\bar{\Pi}_{\text{cur}}, \lceil R \rceil_r),
\end{aligned} \tag{12}$$

which proves (7).

Proof for (8) (write-lock):

If the lock was a write-lock, $v'_{R_m} = 0$. Continuing from (9)

$$\begin{aligned}
\Rightarrow \bar{\Pi} &= \max(\{\bar{\Pi}_{\text{cur}}\} \cup \{\pi(J) \mid 0 < \mu_{R_m}(J)\}) \\
&= \max(\{\bar{\Pi}_{\text{cur}}\} \cup \{\pi(J) \mid J \text{ needs } R_m\}) \\
&= \max(\bar{\Pi}_{\text{cur}}, \lceil R \rceil_w),
\end{aligned} \tag{13}$$

proving (8).

VI. READERS-WRITER LOCK IMPLEMENTATION IN RTIC

Read and write accesses need to be treated as distinct from each other. In effect, two ceilings per resource R are required:

- reader ceiling $\lceil R \rceil_r$: maximum priority among jobs with *write access* to the resource, and
- writer ceiling $\lceil R \rceil_w$: maximum priority among jobs with *read or write access* to the resource.

The protocol bindings and necessary analysis can be provided by a module (“rw-pass”) implementing the read-lock user API and a pre-compilation pass. As the API, a method should be provided with the signature `read_lock(Fn(&T) -> R)`. The method may be implemented simply by calling the conventional `lock` method with the ceiling set to $\lceil R \rceil_r$. As the closure argument, the method should pass a shared, immutable reference to the underlying data structure. Since the resource ($\&T$) is exposed to user code as a shared reference, user code is required by the compiler to conform to the rules concerning shared references, i.e., reads only.

During pre-compilation `rw-pass` should:

- identify all jobs with write-access to each resource R , compute $\lceil R \rceil_r$ according to its definition, and
- transform all DSL read accesses to binary-semaphore-based locks with ceiling set to $\lceil R \rceil_r$.

The main DSL compilation takes as input the DSL with knowledge of all accesses to shared resources. The implementation will now take into account all accesses (both read and write) when computing the ceiling $\lceil R \rceil_w$. This way, no additional target specific code generation is required, as the target specific `lock` implementation can be reused.

VII. FUTURE WORK

Implementation details for the analysis and code generation is left as future work.

With the current implementation, write access code will be generated for resources that are technically read-only. From a safety perspective this is perfectly sound, as the computed ceiling value $\lceil R \rceil$ does not differentiate between

accesses. However, from a modeling perspective rejecting write accesses to jobs with read only privileges would be preferable. Strengthening the model is out of scope for this paper and left as future work.

For general multi-unit resources, the new system ceiling value is different for each number of remaining resources. An overhead-free implementation has not been identified, and the viability of general multi-resource support for RTIC is left as future work.

VIII. CONCLUSION

We have shown that for each read or write lock-operation, a distinct ceiling value can be computed at compile time, and a readers-write lock can be implemented in RTIC at similar cost to the corresponding single-unit/mutex lock. The declarative model can be enforced using Rust ownership rules. The readers-write lock can be implemented as compiler pass in RTIC eVo.

REFERENCES

- [1] T. P. Baker, “Stack-Based Scheduling of Realtime Processes,” technical report, 1990. [Online]. Available: <https://www.cs.fsu.edu/~baker/papers/mstacks3.pdf>
- [2] L. Sha, J. P. Lehoczky, and R. Rajkumar, “Task scheduling in distributed real-time systems,” *IECON’87: Automated Design and Manufacturing*, vol. 857, pp. 909–917, 1987.
- [3] N. Audsley, A. Burns, M. Richardson, K. Tindell, and A. J. Wellings, “Applying new scheduling theory to static priority pre-emptive scheduling,” vol. 8. IET, pp. 284–292, 1993.
- [4] L. Sha, R. Rajkumar, J. Lehoczky, and K. Ramamritham, “A Real-Time Locking Protocol (Read-or-Write Priority Ceiling Protocol),” technical report, 1989. [Online]. Available: <https://ieeexplore.ieee.org/document/83617>
- [5] R. Rajkumar, L. Sha, and J. P. Lehoczky, “Real-Time Synchronization Protocols for Multiprocessors,” in *RTSS*, 1988, pp. 259–269.
- [6] T. P. Baker, “Stack-based scheduling of realtime processes,” *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991.
- [7] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, vol. 24. in Real-Time Systems Series, vol. 24. Springer, 2011.
- [8] The Rust Project Contributors, “The Rust Reference.” 2025.
- [9] H. Lunnikivi, Z. Madaoui, P. Dzialo, and P. Lindgren, “Modular RTIC: Lightweight real time for customized architectures,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2025.
- [10] J. Eriksson, F. Håggström, S. Aittamaa, A. Kruglyak, and P. Lindgren, “Real-time for the masses, step 1: Programming API and static priority SRP kernel primitives,” in *2013 11th IEEE 8th International Symposium on Industrial Embedded Systems (SIES)*, IEEE, 2013, pp. 110–113. doi: 10.1109/SIES.2013.6601482.
- [11] ARM Limited, “ARMv6-M Architecture Reference Manual.” [Online]. Available: <https://developer.arm.com/documentation/ddi0419/>
- [12] ARM Limited, “ARMv7-M Architecture Reference Manual.” [Online]. Available: <https://developer.arm.com/documentation/ddi0403/>
- [13] ARM Limited, “ARMv8-M Architecture Reference Manual.” [Online]. Available: <https://developer.arm.com/documentation/ddi0553/>
- [14] RISC-V International, “The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture, Document Version 20250508: Ratified.” [Online]. Available: <https://docs.riscv.org/reference/isa/attachments/riscv-unprivileged.pdf>
- [15] “Core-Local Interrupt Controller (CLIC) RISC-V Privileged Architecture Extensions (Unratified).” [Online]. Available: <https://github.com/riscv/riscv-fast-interrupt>
- [16] P. Lindgren, P. Dzialo, and H. Lunnikivi, “Hardware support for static-priority stack resource policy based scheduling,” in *2023 IEEE 32nd International Symposium on Industrial Electronics (ISIE)*, 2023, pp. 1–5.